Optimizing Model Performance: The Significance of Missing Data Analysis and Imputation

NAM LE[1], MINH NGUYEN[2]
[1]KHDL2024, FACULTY OF INFORMATION SCIENCE AND ENGINEERING, UNIVERSITY OF INFORMATION TECHNOLOGY, HO CHI MINH CITY, VIETNAM(24521104@GM.UIT.EDU.VN
[2]KHDL2024, FACULTY OF INFORMATION SCIENCE AND ENGINEERING, UNIVERSITY OF INFORMATION TECHNOLOGY, HO CHI MINH CITY, VIETNAM(24521075@GM.UIT.EDU.VN

ABSTRACT In the pipeline of building real-world machine learning models, missing data ($missing\ values$) is a classic challenge that directly impacts the training process and distorts the original data distribution. This project was conducted to deeply investigate statistical missingness mechanisms (MCAR, MAR, MNAR) while empirically evaluating the effectiveness of handling methods ranging from simple to complex: from deletion, statistical imputation (SimpleImputer), multivariate imputation by chained equations (MICE), to advanced deep learning architectures such as DAE or GAN. Our practical experiments on both synthetic and real-world datasets (Horse Colic, World Bank) indicate that there is no "silver bullet" tool. Complex models like MICE or KNNImputer are occasionally prone to overfitting or high computational costs, whereas SimpleImputer demonstrates superior stability and better generalization capabilities on real-world data. To discover more optimal solutions in production-grade environments, the team thoroughly analyzed and re-implemented the source codes of top-tier solutions from major Kaggle competitions, including the Tabular Playground Series, WiDS Datathon, and SIIM-ISIC Melanoma Classification. Practical lessons from domain experts reveal that instead of merely relying on out-of-the-box libraries for imputation, integrating Feature Engineering—such as creating binary missingness indicators ($Missing\ Indicators$) to treat data gaps as predictive signals—is the ultimate key to achieving breakthroughs in model performance. Beyond merely optimizing quantitative metrics, the greatest takeaway our team gained from this project is a transparent mindset regarding the true nature of data. The team clearly recognizes the substantial gap between academic theory from international papers and real-world data processing, thereby drawing a vital lesson in flexibility and proactivity when applying machine learning techniques to ensure models are both highly efficient and explainable.

I. PROBLEM STATEMENT

A. THE PHENOMENON OF MISSING DATA (MISSING VALUES)

In practical research and data processing, missing data (missing values) is the phenomenon where attribute values of one or more observations are not recorded. Systems typically represent this state using characteristic symbols such as NaN, NULL, or special conventional value codes.

B. CONSEQUENCES OF MISSING DATA

If data gaps are not scientifically handled, they lead to the following severe consequences:

- **Disruption of the model training process**: Most current machine learning algorithms (typically the `Scikit-learn` library) do not support directly processing variables containing NaNs, causing system errors during the training phase.
- **Result Bias**: Applying inappropriate handling methods easily alters the original data distribution, thereby causing bias in statistical analysis results and reducing the predictive model's accuracy.
- **Information Loss**: Extreme removal methods (deleting entire rows or columns containing missing values) shrink the sample size, leading to the risk of losing crucial information that constitutes the model.

C. PROJECT OBJECTIVES AND CONTRIBUTIONS

To address the aforementioned limitations, this project focuses on deeply researching the technical foundations of imputation methods, while conducting empirical analysis through specific problems. Specifically, the core objective of this project is to investigate how machine learning experts and top-tier practitioners handle missing data in real-world scenarios, by analyzing winning solutions from past

competitions on the **Kaggle** platform. By dissecting these high-stakes competitions, the project aims to bridge the gap between theoretical imputation techniques—such as statistical baselines, advanced iterative algorithms, or deep learning approaches—and their practical, optimized deployments. The analysis will focus on identifying the decision-making workflows, feature engineering tricks, and model-specific imputation strategies that consistently yield superior predictive performance under complex missingness mechanisms.

II. MISSING DATA ANALYSIS PROCESS

A. THEORETICAL FOUNDATION

1) Classification of Missing Data Origins

In statistical theory, the generating mechanism of missing data is classified into three basic models based on the linear or non-linear dependence between the missing probability and the data matrix:

- **Missing Completely At Random (MCAR)**: The missing phenomenon occurs absolutely randomly. In other words, the probability of an observation being missing is completely independent of both the observed variables and its own actual value. This is the most ideal assumption, simplifying statistical inference methods without altering the population's original distribution.
- **Missing At Random (MAR)**: The probability of missing data depends only on the available information (variables already observed and recorded in the system) but is completely independent of the unobserved, hidden value of that variable itself. For this mechanism, bias can be controlled through appropriate preprocessing techniques based on the underlying data.
- **Missing Not At Random (MNAR)**: This is the most complex mechanism, occurring when the missing probability directly depends on the very value of the missing variable (or uncollected latent variables). Handling the MNAR mechanism requires specialized assumed models and complex structures to estimate the data generation behavior, as the sample's distribution no longer represents the original population.

2) Handling Methods for Each Type of Missing Data

Handling methods into main categories, ranging from classical to modern:

- Deletion: Listwise and Pairwise Deletion.
- Statistical-based Imputation: Single Imputation (Mean, Median) and Multiple Imputation.
- ML-based Imputation: Algorithm groups including Regression, KNN, Tree-based, SVM, and Clustering.
- NN-based Imputation: Application of generative models such as Variational Autoencoders (VAE), Generative Adversarial Networks (GANs), and the latest trend, Diffusion Models.
- Representation Learning: Utilizing Graph Neural Networks (GNN) to capture complex structural relationships between missing data points.

TABLE 1. Traditional Methods (Deletion & Statistical)

Method Category	Typical Techniques	Advantages	Disadvantages	Suitable Mechanism
Deletion	Listwise, Pairwise	Simple to implement, extremely fast computation.	Deletes useful information, causes strong distribution bias.	MCAR only.
Statistical	Mean/Median, Multiple Imputation	Easy to apply, maintains sample size.	Distorts variance, completely ignores cross-feature correlation.	MCAR, partially MAR.

TABLE 2. Traditional Methods (Deletion & Statistical)

Typical Technique	Data Recovery Mechanism	Strengths	Limitations	Suitable Mechanism
KNN-based	Finds k nearest observations (neighbors) to impute the value.	Highly effective with non-linear data.	High computational cost when data scales up, sensitive to noise.	MCAR, MAR.
Tree-based	Uses Tree algorithms (Random Forest, XGBoost) for prediction.	Excellent at handling cross-correlations and categorical data.	Prone to overfitting if the feature's missing rate is too high.	MAR, potential for mild MNAR.

TABLE 3. Traditional Methods (Deletion & Statistical)

Architecture	Operating Mechanism	Special Processing Capabilities	Suitable Mechanism
GANs & VAEs	Generates missing data by learning the latent space distribution of the original data.	Excellently reconstructs complex distribution structures where traditional ML fails.	MAR, MNAR.
Diffusion Models	Recovers original data from noise through step-by-step decoding.	SOTA-level accuracy in generating locally missing tabular data.	MAR, MNAR.
GNNs (Graph)	Represents the dataset as a Graph to exploit topology.	Discovers hidden links between clustered missing records.	Specialized for MNAR.

B. MISSING DATA HANDLING TECHNIQUES IN THE SKLEARN LIBRARY AND THE SKLEARN.IMPUTE PACKAGE

1) Univariate Imputation - SimpleImputer

SimpleImputer is a tool for univariate missing data imputation using basic statistical metrics.
Hyperparameters:

- "strategy": `"mean"`, `"median"` (robust to outliers), `"most_frequent"` (`"mode"` - used for categorical data), or `constant` (a fixed value).
- "missing_values": The placeholder for missing data (default is `"np.nan"`).
- "add_indicator": True/False (creates an additional binary column to flag originally missing locations).

2) Multivariate Imputation

a: ***Definition:***

MICE (Multivariate Imputation by Chained Equations) is a method for handling missing data by:

- Initializing missing values (using mean/median).
- Building a predictive regression model for each column.
- Updating values through multiple iterations until convergence.

Parameters: `"max_iter"`, `"estimator"` (BayesianRidge, RandomForest, ExtraTrees), `"initial_strategy"`, `"imputation_order"`

b: ***Testing on Synthetic Data***

Testing 6 imputation configurations across 3 missing data mechanisms:

- MCAR (Missing Completely at Random)
- MNAR (Missing Not at Random)
- Combined

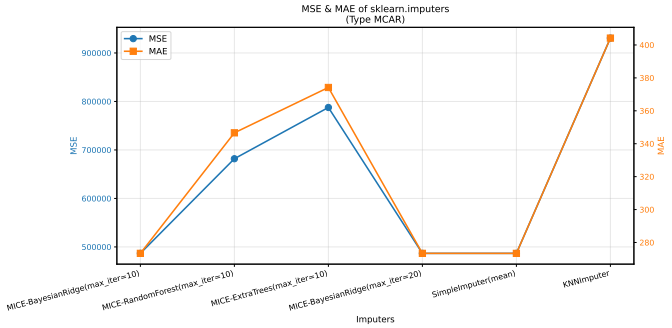

FIGURE 1. MCAR errors chart

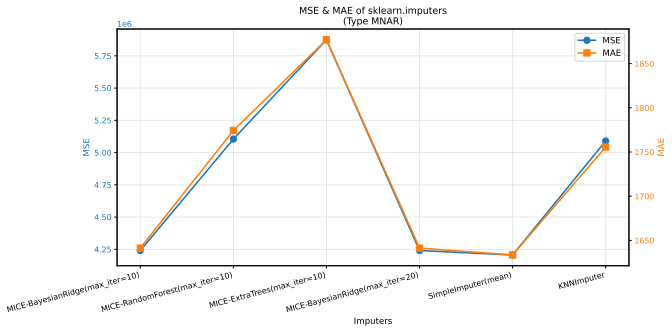

FIGURE 2. MNAR errors chart

Conclusion: BayesianRidge delivers the best performance (lowest MSE/MAE).

c: ***Testing on Real Data:***

Horse Colic Dataset *

- Comparison of 6 imputation methods
- Predict churn using RandomForestClassifier with 5-fold cross-validation.
- Metric: F1-score, Accuracy
- Results: MICE methods yield comparable outcomes.

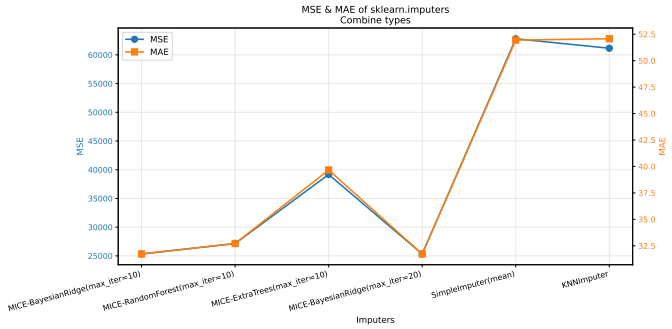

FIGURE 3. Combined type errors chart

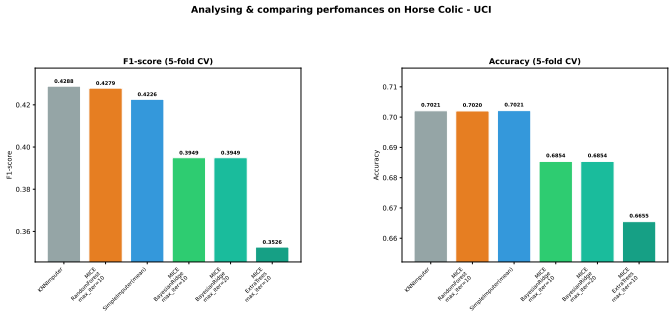

FIGURE 4. Prediction scores on Horse-colic dataset (preprocessed)

World Bank Dataset *

- Use RandomForestRegressor with 5-fold CV
- Metric: MAE, MSE
- Results: Evaluating imputation performance on economic data.

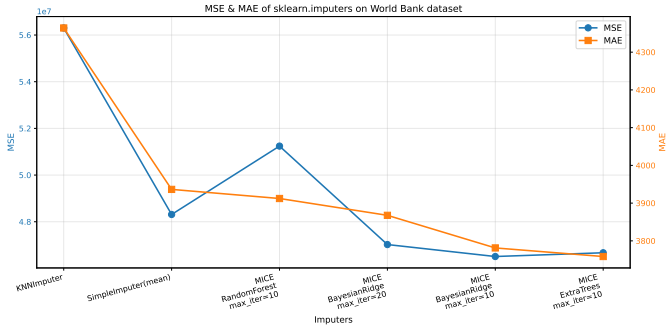

FIGURE 5. Prediction errors on World-bank dataset (preprocessed & logically imputed)

* *These two datasets have been uploaded to the attached Folder (as the World Bank dataset was manually compiled by the team rather than being publicly available online).*

3) **KEY FINDINGS**

a: ***MICE-BayesianRidge is not the silver bullet we expected:***

TABLE 4. Traditional Methods (Deletion & Statistical)

Scenario	Best Method	Notes
Synthetic MCAR	MICE-BR = SimpleImputer	Comparable performance
Synthetic MNAR	SimpleImputer	Unexpected result!
Synthetic Mixed	MICE-BayesianRidge	Only performs well here
Horse Colic	SimpleImputer	MICE-BR performs worst
World Bank	SimpleImputer	MICE-BR performs worst

Explanation: MICE performs well only when:

- There are sufficient samples relative to features (not severely sparse).
- Missing patterns are complex (involving many-to-many relationships).
- Data is not excessively high-dimensional.

b: ***SimpleImputer consistently outperforms other methods on real-world data.***

Explanation: For real-world tabular data, SimpleImputer often offers the best trade-off between:

- Model complexity
- Risk of overfitting
- Computational cost

TABLE 5. Traditional Methods (Deletion & Statistical)

Dataset	Sample-to-Feature Ratio	Is SimpleImputer Justified?
Horse Colic	$299/58 = 5.1$	✓Low ratio → SimpleImputer is reasonable
World Bank	?	✓Economic data → Simple model is sufficient
Synthetic (1000/4)	250	✓MICE performs better

c: ***ExtraTrees consistently performs the worst among all MICE variants.***

- Synthetic MCAR: ExtraTrees underperforms by 62% compared to BayesianRidge.
- Synthetic Mixed: ExtraTrees underperforms by 55
- Never achieves the best performance.

Explanation: ExtraTrees utilizes random splits with deep trees → struggles to capture loose data relationships → results in poor imputation.

d: ***KNNImputer is not a lifesaver here.***

Consistently performs worse than or equal to SimpleImputer. Synthetic MCAR: MSE is 1.9 times higher. World Bank: MSE is 1.25 times higher. Explanation: KNN struggles with: High-dimensional data (curse of dimensionality). Complex missing patterns (neighbors may also have missing data).

> Research findings indicate that no single method outperforms all others across all scenarios. *SimpleImputer* dominates on real-world data due to its stability and generalization capabilities, whereas *MICE-BayesianRidge* shines when the data is sufficiently rich and the missing patterns are distinct.

NOTE: Meanwhile, experiments aimed at learning and tool utilization have not been presented in detail within this report, but are summarized in the attached Folder

III. CASE STUDIES OF KAGGLE COMPETITIONS

A. TABULAR PLAYGROUND SERIES (JUNE 2022)

1) Competition Description

The June 2022 Tabular Playground Series revolves entirely around data imputation. This dataset is similar to the May 2022 edition, except there is no target variable. Instead, the dataset contains missing values, and the objective is to predict what those values are.
Link: https://www.kaggle.com/competitions/tabular-playground-series-jun-20
data?select=data.csv
Access Date: 24/05/2026

2) Dataset Description

Scale: Synthetic dataset with 1,000,000 rows and 80 columns (features). Feature Structure: All data is anonymized and divided into 4 main groups:

- `"F_1"` (15 columns) and `"F_3"` (25 columns): Continuous numerical data.
- `"F_2"` (25 columns): Discrete/categorical numerical data, containing no missing values.
- `"F_4"` (15 columns): Continuous numerical data. This is the most critical feature group due to strong correlations and complex non-linearity.

Missing Data Characteristics:

- Missing values only occur randomly in groups `"F_1"`, `"F_3"`, and `"F_4"`.
- The missing rate is not massive (about 1.8 % of the total data) but is scattered across many rows.

TABLE 6. Distribution of the number of missing values for F_4 per row

Number of missing values (NaNs)	Number of rows
0	759,268
1	211,342
2	27,127
3	2,124
4	135
5	4

TABLE 7. Average variance of each group

Group	Mean variance
Nhóm F_1	0.9063
Nhóm F_2	2.3916
Nhóm F_3	0.9614
Nhóm F_4	4.8759

Evaluation Metric: RMSE (Root Mean Squared Error) calculated based on the error between the imputed values and the actual values hidden by the organizers.

3) Top Solutions' Methodologies (Published)

a: Top 1

Mask Generation for Simulating Missing Data:

- `"random_mask(...)"`: Used for training, creating random missing data at various levels.
- `"mask_n_rows(...)"`: Used for validation, fixing the number of masked cells to compare models fairly.
- Idea: Teach the model how to reconstruct data under various missing scenarios.

Data Standardization:

- Used StandardScaler to scale F_4 columns.
- Missing values were replaced by 0 after scaling using "np.nan_to_num(...)" before being fed into the network.

Building a DAE (Denoising Autoencoder) Model:

- The model is a MaskedAutoencoder.
- Architecture includes "inp_proj" (embedding for input values), "mask_proj" (embedding for the missing mask), multiple MLP layers with skip connections, and final_dense (re-predicting all output columns).

Specialized Loss Function: MaskedMSELoss only calculates errors on the masked positions in the original data. The model is not penalized for relearning already observed cells, focusing solely on the parts that need imputation.

b: Top 2

"F_1" and "F_3" Groups: Handled missing values using Mean Imputation. "F_4" Group: Applied a complex multivariate prediction method:

- Transformed the problem by sequentially using a missing column as the target variable and the remaining columns as input features.
- Decomposed the F_4 dataset into 6 independent groups based on the number of NaNs per row: from 0 NaNs (complete) to 5 NaNs.
- Used the 0NaN group as the training set, while the remaining 5 groups acted as the predicting set for imputation.
- Trained custom models for each xNaN group by dynamically changing the output dimensions of the target variable x.

c: Top 4

"F_1" and "F_3" Groups: The final submission was an ensemble of Linear Models and Descriptive Statistics of these features themselves. "F_4" Group:

- The chosen approach utilized a Simple Dense Neural Network, implemented via Keras and TensorFlow.
- Simultaneously, the solution fine-tuned another candidate model specifically for "F_4", derived from a public notebook titled "TPS Jun 2022 PyTorch Lightning with NA counts".

4) Extracted Insights

Typically, DAE (Denoising Autoencoder) is known for image or audio signal denoising tasks. The insight here is flexibility in thinking: treating missing data cells (NaNs) as a form of "noise," thereby successfully applying DAE to tabular data imputation. The author immediately chose an Autoencoder as the baseline to avoid looping through and predicting each feature individually. Instead of using traditional models (like XGBoost/LightGBM) to predict each missing column discretely, DAE allows learning the distribution of the entire dataset and interpolating all columns simultaneously. The author emphasized: Using a standard Autoencoder alone "performed quite poorly." The difference in the Top 1 model lies in persistently applying architectural tricks: It is mandatory to have an embedding or attention mechanism for the missing mask to force the neural network to concentrate entirely on accurately predicting NaN locations rather than just copying data.

B. WIDS DATATHON 2021

1) Competition Description

Link: https://www.kaggle.com/competitions/widsdatathon2021/data?select=UnlabeledWiDS2021.csv
Access Date: 24/05/2026
Problem Type: Binary Classification.
Target Variable: "diabetes_mellitus" (1: Patient diagnosed with diabetes, 0: Not diagnosed).

TABLE 8. Top 10 missing percentage columns (%)

Feature	Missing Percentage (%)
h1_bilirubin_min	92.089553
h1_bilirubin_max	92.089553
h1_albumin_min	91.431886
h1_albumin_max	91.431886
h1_lactate_min	91.018539
h1_lactate_max	91.018539
h1_pao2fio2ratio_min	87.123243
h1_pao2fio2ratio_max	87.123243
h1_arterial_ph_max	82.860699
h1_arterial_ph_min	82.860699
dtype: float64	
→ Total of missing features > 80% features: 32 columns.	

Context: Predicting whether a patient admitted to the Intensive Care Unit (ICU) has a history of diabetes, using only clinical data collected within the **first 24 hours of admission.**

2) Dataset Description

Number of Features: 181 variables. Data includes major groups:

- Demographics: Age, gender, BMI, ethnicity, weight, height.
- ICU Information: ICU type, hospital ID, admission source.
- Vital Signs: Heart rate, systolic/diastolic blood pressure, respiratory rate (min/max values in 24h).
- Lab Results: Glucose levels, hemoglobin, calcium, potassium...

A few notes on the features:

- "Vital sign" features are collected continuously by machines attached to the patient's arm. However, since updating data at such a high frequency would cause the dataset size to explode, these features have been aggregated by calculating their minimum and maximum values.
- For 'Lab Results' features, instead of using continuous values directly, they are grouped into discrete "bins" to calculate risk scores.
  For example, with Glucose in the scoring system:
  - < 40 mg/dL: +8 points
  - 40 - 60 mg/dL: +4 points
  - 61 - 250 mg/dL: 0 points (Normal)

Sample Size:

- Train set: 130,157 records.
- Test set: 10,234 records.

Main Dataset Issues:

- Massive Missing Values: Many lab/vital columns are missing over 80% or 90%. Missing data belongs to Missing Not At Random (MNAR):
  - Time constraints ("h1_ prefix"): "h1_" data only captures the first hour of admission. Unlike vital signs which are immediately available via monitors, blood tests require a 1-2 hour process, making it extremely rare to have results within this timeframe.
  - Selective clinical indication: Doctors do not order blind testing for everyone but base it on patient symptoms. For example, Lactate is only used when shock or sepsis is suspected; Bilirubin and Albumin are not top priorities in life-threatening emergencies.
  - High invasiveness: For instance, pao2 requires arterial blood gas sampling, which is painful and carries risks. Therefore, it is only ordered for patients with severe respiratory issues, while the majority of others are monitored using non-invasive methods (SpO2).

- Severe Class Imbalance: In the training set, only 21.6% of the rows have the "diabetes_mellitus" feature equal to 1 (i.e., having the disease).

3) Top Solutions' Methodologies (Published)

TABLE 9. Summary of Top Solutions Methodologies

Author	Val.	Feature Eng.	Model	Missing Val.	Preprocess.
Top 1	Rand 5-Fold.	TE, group mean, min / max flags, dec. count.	3-Tree Stack.	Count missing; Native handle.	Sim ICU overlap.
Top 2	K-Fold (Adv Val).	Freq enc, interact, aggr, OHE.	LGBM Ens (3 seeds).	Native handle.	Adv sample filter.
Top 3	5-Fold Strat.	Min-max mean / range, ratios.	120 models (5 algos).	2-Stage: KMeans + global.	Groupby - Agg - Drop.
Top 4	CV.	Binning, stats, freq exploit.	2-Layer Stack.	Iter LGBM.	Min-max clip, Rank norm.
Top 9	Adv Val (Drift).	Math combo, TE, missing ind.	LGBM + CatBoost.	MICE (w / h).	Fix min / max, Drop BP, Cluster.
Top 10	CV.	"2020features", Label Enc.	CatBoost + LGBM.	N/A.	Std Scaler, SHAP.

a: Top 1

Data Splitting Strategy: 5-Fold cross-validation without stratify. The authors chose random K-Fold to accurately simulate the ICU ID overlap between the Train and Test sets. Allowing the same ICU ID to appear in both the training and validation folds effectively utilizes ICU ID-based features and ensures the validation score closely reflects the actual test evaluation.

Feature Engineering:

- The authors performed Feature Importance analysis and identified that the most significant features are vital signs, such as "d1_sysbp_noninvasive_max" and "d1_sysbp_ooninvasive_min"
- Created a flag for when min and max readings are equal indicating the patient wasn't critical enough for multiple measurements.
- Grouping: Clustered patients with the same diagnosis and calculated the mean of their measurements.
- Created a feature representing the distance between a patient's metrics and the group mean.
- Target encoding for "apache_2_diagnosis" and "apache_3j_diagnosis" (high-cardinality categorical features).
- Target encoding for interactions of categorical features.
- Created a feature counting decimal places to represent a latent feature.
- Handling Data Clipping: Created a feature for min/max values to help the model recognize potential outliers handled by institutional data clipping (e.g., temperatures capped at a ceiling value).

Missing Value Handling:

- Proactive Feature Engineering: Did not impute with mean/median. Instead, counted the number of missing values to create a new feature, assuming "not measured" is a predictive signal.
- Native Model Handling: Relied on XGBoost and LightGBM's built-in ability to automatically handle null values by calculating the optimal split direction for blank cells during training.

Modeling: Stacking with 3 tree-based models.

b: Top 2

Feature Engineering: Calculated category frequencies with log1p, created interaction variables, grouped aggregate features, and converted text to one-hot/label encoding.
LightGBM handled NaNs natively without prior imputation. **Adversarial Validation:**
Merged test and train sets with pseudo labels to train an LGBM to distinguish them. Used KFold to extract probabilities ("y_preds") to evaluate train-to-test similarity.
Sample Filtering: Filtered out a portion of the train data based on "y_preds.argsort()" to create a training set closer to the test distribution.
Main Training: Trained an LGBM ensemble multiple times (3 seeds) on the filtered train set; merged predictions using the geometric mean.

c: Top 3

Data Splitting Strategy: 5-fold Stratified Cross Validation.
Feature Engineering: For min-max features, created mean (min + max) / 2 and range (max - min) features.

Handling Missing Values: Iterative imputation via LGBM failed (only imputed age, weight, height). High missing rates (60-80 %) meant the model lacked information to learn cross-feature relationships, hitting early stopping thresholds without reducing error.
Two-Stages:

- Stage 1: Clustered data with KMeans using height, weight, and age. Applied simple mean imputation on each cluster separately.
- Stage 2: Fixed remaining global missing values via overall mean-imputation

Ratio Features: Calculated ratios to keep the model unbiased. Resolving HighCardinality Overfitting: Used Adversarial Validation to discover the train/test gap relied heavily on ICU_id. Forced the model to learn the ICU context rather than its ID via Groupby -> Aggregate -> Drop.
Modeling: 120 models across 5 algorithms (Catboost, XGBoost, LightGBM, Tabnet, Logistic Regression). Used Optuna for hyperparameter tuning.

- Forced optimization into 3 complexity levels (Shallow, Medium, Deep) instead of free searching.
- Used a "draft" training approach: Ran short epochs, eliminated bad configs early, and fully trained only the promising ones.

d: Top4

The team's data processing pipeline focuses on two main stages to build the LGBM model (with ∼1,300 features, achieving a score of 0.87333):

Feature Extraction:

- **Creating new features:** Binning data and calculating summary statistics to generate highly specific and complex numerical features (e.g., "d1_creatinine_mean_dist_of_std_from_d1_glucose_shifted_diff_bin_mean").
- **Exploiting measurement frequency:** Leveraging the variations in the number of patient measurements to:
  - Create categorical features by counting the number of distinct values in the min, max, and apache columns.
  - Mitigate discrepancies caused by coincidence in columns like "max_glucose" by realigning data distributions (adding offsets) based on measurement frequency.

Feature Engineering:

- **Missing data imputation:** Utilizing an iterative LGBM regressor model instead of basic imputation methods.

- **Data synchronization:** Applying clipping to constrain numerical features in both train and test sets to share the same min-max range.
- **Feature selection:** Iteratively dropping one of the two highly correlated variables based on Spearman correlation to eliminate redundant information.

Modelling Layer 1 (Base Models): Trained 127 diverse models (LGBM, XGBoost, TabNet, Random Forest, etc.) with varying hyperparameters, encoding techniques, and imputation methods. Manually selected the 40 least correlated predictions to pass to the next layer. Layer 2 (Meta Models): Utilized KNN, Logistic Regression, TabNet, LGBM, and XGBoost. This layer combined Layer 1 predictions with newly calculated meta-features, entirely excluding the original features. Optimization Techniques:

- Applied rank normalization to predictions across models and folds to ensure stability and preserve the correlation between Cross-Validation (CV) scores and actual leaderboard scores.
- Experimented with Test-Time Augmentation (TTA), though it yielded no independent improvement.

e: Top 9

Data Cleaning

- Logical inconsistency handling: Flipped min and max values in cases of inequality errors (min > max).
- Data distribution synchronization: Removed records with age equal to 0 or missing from the train set to align with the test set's data characteristics.
- Missing data imputation: Applied MICE (Multivariate Imputation by Chained Equations) to fill missing values for weight and height based on age (performed independently by gender), then recalculated BMI.
- Multicollinearity reduction: Dropped general blood pressure features that did not specify the measurement method (invasive/non-invasive), including mbp, sysbp, and diasbp, to reduce information redundancy due to their extremely high correlation with the corresponding non-invasive features.

Feature Engineering

- Baseline mathematical features construction: Created interaction features in the form of differences and ratios, including max-min, max/min, d1-h1, d1/h1, sysbp-diasbp, and average features (min+max)/2.
- High-order combined features development: Generated complex 2nd-level combination features to deeply exploit relationships between variables (e.g., (d1 max - d1 min) - (h1 max - h1 min)).
- High-cardinality categorical encoding: Applied Target Encoding to system identifier variables like `"icu_id"` and `"apache_2_diagnosis"`.
- Missing Indicators initialization: Created binary variables to flag the missing data status for d1/h1 test results.
- Binning Profiling: Binned the age and bmi variables. Combined age, bmi, ethnicity, and gender to create patient profiles. Binned blood pressure and glucose indices based on clinical risk factors.
- Aggregation features integration: Calculated the mean of mathematical features, then aggregated them by profile, `"apache_3j_diagnosis"`, and blood pressure/glucose bucket.
- Frequency Encoding: Applied Frequency Encoding to all remaining categorical variables in the dataset.

Feature Selection

- Methodology: Built a baseline CatBoost model to calculate Permutation Importance. Subsequently applied Hierarchical Clustering to group highly correlated variables together.
- Screening criteria: Removed variables with negative importance, along with variables with near-zero positive importance if they were in a single-element cluster (cluster size of 1). This process was repeated iteratively until the feature set stabilized.
- Dataset Shift evaluation: Used an Adversarial Validation model to check the similarity between the train and test sets. The results showed poor separation performance by the adversarial model, confirming that the filtered feature set did not suffer from severe data drift, and thus was kept intact.

Model Building

- Phase 1 (Baseline): Deployed and optimized performance using the CatBoost algorithm as the initial reference framework.
- Phase 2 (Algorithm transition): Shifted development to the LightGBM model, resulting in significant performance score improvements.
- Phase 3 (Ensemble): The final optimal solution utilized model ensembling, combining the best-optimized LightGBM configurations with the best CatBoost models to maximize generalization and achieve the highest accuracy on the dataset.

f: Top 10

Methods that Improved Performance

- Model Selection: AutoML (AutoMLJAR) analysis identified CatBoost and LightGBM (LGBM) as the most effective models for the task.
- Feature Processing: Applied the "2020features" set, incorporated Label Encoding and Standard Scaler, and utilized SHAP analysis for feature selection.
- Optimization and Ensembling: Used Optuna for LGBM hyperparameter tuning. The final predictions were generated by ensembling CatBoost and LGBM.
- Post-processing: Performance gains were recorded by applying the rankdata function (from scipy.stats), averaging multiple output files, and employing power/geometric averaging on highly correlated submissions (yielding an increase of 0.001).

Methods that Did Not Improve Performance Neural Networks: Experiments involving TabNet and other neural network architectures recorded the lowest prediction results compared to the tree-based models in this specific problem.

4) Implementation

Due to the diversity in the authors' validation strategies, the team decided to reproduce the experiments using an incremental approach. Specifically, we established a tree-based baseline model to leverage its native ability to handle missing values. From there, the team sequentially integrated the preprocessing techniques and fine-tuned the models as described by the authors. The objective of this method is to objectively evaluate the improvement in accuracy, as well as to estimate the computational cost and implementation complexity of each technique. The implementation results are presented in the table below:

	Baseline (ROC - AUC)	Add Preprocessing Techniques	Change Models
Top 1	**8.639**	8.657	8.685
Top 2	**8.639**	8.688	8.707
Top 3	**8.639**	8.683	*
Top 4	**8.639**	8.492	8.708
Top 9	**8.639**	**8.543**	*
Top 10	**8.639**	*	*

TABLE 10. Model performance comparison

Rather than focusing on comparing methods to determine which is superior, our team's primary objective was to learn from preceding experts. We dedicated the majority of our time to analyzing and re-implementing effective techniques from top-performing teams.

Despite facing obstacles such as unpublished source codes and strict time constraints, we strove to recreate the most critical processing steps to deeply understand the core of the problem. Consequently, the implementation results did not achieve the breakthrough improvement in ROC-AUC that we had anticipated. This experience made us realize the critical importance of establishing a clear analytical mindset from the outset, rather than merely piling on Feature Engineering techniques or deploying more complex models.

5) Extracted Insights

Competition Knowledge

- Optimization Strategy: A simple baseline provides a solid foundation, but achieving a breakthrough in rankings requires a deep focus on Feature Engineering and the integration of Ensemble models.
- Validation Strategy: There is no one-size-fits-all method. Validation approaches must be chosen flexibly based on evaluation goals and dataset characteristics to prevent biased results (overfitting).

Mindset & Foundational Skills

- Effort Allocation: The core and most time-consuming challenge is understanding the underlying meaning of the data (domain knowledge). Once deeply understood, executing Feature Engineering based on experience becomes rapid.
- Nature of Preprocessing: Real-world data processing is inherently diverse and complex. It demands adaptable and customizable thinking rather than rigid reliance on built-in libraries or predefined algorithms.
- Standard Workflow: Maintain a consistent thought process to refine code and effectively uncover insights. The optimal pipeline should be: Baseline → Feature Analysis → Meaningful Feature Extraction → Feature Engineering (alongside preprocessing) → Model → Fine-tune.

C. TABULAR PLAYGROUND SERIES - AUG 2022

1) Competition description

Link: https://www.kaggle.com/competitions/tabular-playground-series-aug-2022
Access Date: 24/05/2026
Problem Type: Binary Classification
Target Variable: `failure` (0-normal, 1-failure)
Context: The August 2022 edition of the Tabular Playground Series is an opportunity to help the fictional company Keep It Dry improve its main product Super Soaker. The product is used in factories to absorb spills and leaks.
Main Dataset Issues:

- Missing data is scattered across multiple columns
- The main columns with missing data include loading and those from `measurement_3` to `measurement_17`
- The average missing data rate is approximately 2.9%, peaking at about 8.6% in the `measurement_17` column.

2) Dataset description

Scale: 26570 rows and 26 columns The features are divided into three main categories:
Identifiers

- `id`: A unique identifier for each individual product.
- `product_code`: A categorical identifier for the batch or type of product (e.g., 'A', 'B', 'E').

Fixed Product Attributes

- `"loading"`: The amount of fluid (load) absorbed by the product during the simulated experiment.
- `"attribute_0"`, `"attribute_1"`: Categorical variables representing the material or structural composition of the product.
- `"attribute_2"`, `"attribute_3"`: Numerical constants describing fixed specifications of that specific `product_code`.

`"measurement_0"` through `"measurement_17"`: A series of continuous numerical values representing the results of various laboratory testing methods performed on each individual product.
Missing data characteristics

- Overall, the missing data rate is negligible, but it is scattered across multiple columns.
- Missing data occurs in the `"loading"` column and from `"measurement_3"` to `"measurement_17"`
- The average missing rate is around 2.9%, peaking at approximately 8.6% in the `"measurement_17"` column

Evaluation metric: evaluating Area under the ROC curve (AUC) between the predicted probability and the observed target.

3) Top Solutions' Methodologies (Published)

a: Top 1

Data Cleaning: Columns exhibiting inconsistencies in the number of unique values between the training and test sets (e.g., `"attribute_1"`) were removed to avoid potential distribution mismatches and improve model reliability.
Missing Values Handling In this method, the author adopts a basic yet consistent approach:

- Imputation: A SimpleImputer with a `"mean"` strategy is used to fill all missing values (NaN) across the `"measurement"` columns.
- Feature Filtering: The author noticed a discrepancy in unique values between the Train and Test sets (e.g., `"attribute_1"`). To address this, these columns were dropped from the model rather than being retained for processing.

Feature Engineering: No new features were generated. However, Batch Normalization was incorporated at the beginning of the neural network architecture to stabilize training and accelerate convergence.
Feature Selection: Features identified as having inconsistent value distributions between the training and test datasets (e.g., `"attribute_1"`) were excluded from the model input.
Modeling: A Deep Neural Network (DNN) was employed with two alternative architectures:

- **Concat Architecture**: Multiple parallel branches process the input independently through Dense layers, after which their outputs are concatenated and passed to a final Dense layer for prediction.
- **Mean Architecture**: Each branch produces an independent probabilistic prediction using a sigmoid output layer, and the final prediction is obtained by averaging the outputs of all branches using `"tf.reduce_mean"`.

The network architecture was generated recursively, allowing customizable depth and width. Model training used Binary Cross-Entropy as the loss function for the binary classification task. To improve generalization and training stability, **`EarlyStopping`** and **`ReduceLROnPlateau`** callbacks were applied during optimization.

b: Top 8

Data Cleaning: No explicit data cleaning procedures were reported. The primary focus was on handling missing values and refining the feature set rather than removing or correcting data records.
Handling Missing Values: The author employed an **"UberImputer"**
strategy that goes beyond traditional imputation methods:

- Missing values were imputed while simultaneously creating missingness indicator features (`wasMissing`) to capture information contained in the missing-value patterns themselves.
- A specific adjustment was also applied to measurement_2, resulting in a small but measurable performance improvement.
- For the subset of samples with missing loading values (approximately 1% of the dataset), a separate modeling strategy was developed instead of relying solely on imputation.

Feature Extraction: No explicit feature extraction techniques were applied. The approach relied on the original dataset features and additional missingness indicators generated during preprocessing.
Feature Engineering: Several feature engineering techniques were incorporated:

- Creation of missingness indicator variables (`wasMissing`) to encode whether a value was originally missing.
- Integration of these indicators with the original features to provide additional predictive information.
- Minor modifications to specific variables such as `measurement_2` to enhance model performance.

Feature Selection: Feature selection played a central role in the methodology:

- Features were systematically ranked according to their estimated predictive importance.
- High-priority variables included `loading`, `measurement_17`, and `measurement_6`.
- Lower-priority features included missingness indicators (`wasMissing`) and dummy variables.
- This iterative feature refinement process contributed an improvement of approximately 0.0088 in model performance.

Modeling: The primary model was a Logistic Regression classifier with the following characteristics:

- Input features were standardized through feature scaling.
- L1 regularization was applied to encourage sparsity and improve generalization.
- The `liblinear` solver was used for optimization.

In addition, a specialized model was developed for samples with missing loading values:

- The `loading` feature was completely removed for this subset.
- A shallow `LightGBM` classifier with a maximum depth of 2 was trained specifically for these observations.
- Although this approach slightly reduced the public leaderboard score, it substantially improved private leaderboard performance, demonstrating its effectiveness for handling this particular subgroup.

Ensembling: To improve prediction robustness and stability, the author employed a median-based ensemble strategy:

- Ten models were trained on different data splits using the **ProductSplitter3v2** framework.
- An additional model was trained on the full dataset.
- This resulted in 11 separate predictions for each test sample.
- The final prediction was obtained by taking the median of these predictions rather than the mean.
- This aggregation strategy reduced prediction noise and improved the score by approximately 0.00218.

c: Top 9

Handling missing values The author focuses on handling missing values (NaN) in the dataset. The process involves identifying columns with missing data and applying imputation techniques to ensure the model does not encounter errors during training:

- Using KNN Imputer (K-Nearest Neighbors): Instead of simply filling in the mean, the author utilized a KNN Imputer
- Missing Indicator Strategy: Similar to the approach used by top competitors, the author cleverly created dummy variables to mark missing locations (`"_missing"` or `"_was_missing"`). The goal was to treat 'missing data' not as an error to be removed, but as an informative signal.

Feature Engineering

- Encoding: The author handles categorical variables (such as `"product_code"` or material attributes) using appropriate encoding methods to convert them into numerical format, helping the model understand the structural attributes of the products.
- Scaling: The author performs feature scaling to ensure all measurement variables are on the same scale. This is critical for distance-based algorithms (or linear models) to function stably.

Model Training Strategy Rather than relying on a single architecture, the author implemented a highly systematic model training strategy:

- **Validation Strategy**: The author systematically splits the dataset into training and validation sets to evaluate model performance before predicting on the unseen test set.
- **Modeling**: Based on the notebook code, the author utilizes libraries like pandas and scikit-learn (or equivalents) to build the model. The training pipeline follows these steps: model initialization -> training on the Train set -> evaluation on the Validation set -> prediction fine-tuning.

4) Extracted Insights

Handling Missing Data: Top performers moved beyond simple mean imputation by utilizing "Missing Indicators" (binary variables), treating missingness as an informative signal.
Feature Engineering: Strategic feature selection and dropping columns with distribution shifts between training and test sets outperformed exhaustive imputation.
Modeling Strategy:

- Ensembling: Median-based ensembling was preferred over averaging to reduce prediction noise and outliers.
- Specialized Sub-models: Training lightweight models (e.g., LightGBM) specifically for data subsets with missing critical features yielded significant gains on the private leaderboard.

Training Robustness: Stable pipelines using cross-validation, Early Stopping, and learning rate scheduling were standard practices to prevent overfitting.

D. SIIM-ISIC MELANOMA CLASSIFICATION

1) Competition description

Link: https://www.kaggle.com/competitions/siim-isic-melanoma-classification/leaderboard
Access Date: 24/05/2026
Problem Type: Binary Classification
Target Variable: `target` (0 denotes benign, and 1 indicates malignant.)
Context: The competition, organized by SIIM and ISIC, aims to develop AI models capable of detecting melanoma from dermoscopic skin lesion images. In addition to image data, patient-level contextual information is provided to assist diagnosis. The broader goal is to support dermatologists in clinical decision-making and improve early skin cancer detection.
Main Dataset Issues:

- Significant class imbalance, with melanoma cases representing only a small proportion of the dataset.
- High visual similarity between malignant and benign skin lesions, making classification challenging.
- Variability in image acquisition conditions, including differences in lighting, resolution, skin tone, and imaging devices.

- Potential patient-level correlations, where multiple images may originate from the same patient, increasing the risk of data leakage if not handled properly.
- Need to effectively combine image features with patient metadata to maximize predictive performance.

2) Dataset description

The dataset consists of medical images in DICOM format, with additional variants in JPEG format, and includes associated metadata. Scale: 33,126 observations and 8 features Columns

- `"image_name"` - A unique identifier pointing to the filename of the corresponding DICOM image."
- `"patient_id"` - unique patient identifier
- `sex` - the sex of the patient (when unknown, will be blank)
- `"age_approx"` - approximate patient age at time of imaging
- `"anatom_site_general_challenge"` - location of imaged site
- `"diagnosis"` - detailed diagnosis information (train only)
- `"benign_malignant"` - indicator of malignancy of imaged lesion
- `"target"` - binarized version of the target variable, with value 0 denotes benign, and 1 indicates malignant.

Missing Data Characteristics:

- Overall, the missing data rate is negligible, scattered across the following columns: `"sex"`, `"age_approx"`, and `"anatom_site_general_challenge"`.
- The highest missing rate is in the `"anatom_site_general_challenge"` column (1.59%).

Evaluation metric: Area under the ROC curve (AUC) between the predicted probability and the observed target.

3) Top Solutions' Methodologies (Published)

Summary of Top Solutions Methodologies

a: Top 1

Data Cleaning: The author performed label normalization to reduce noise in the training data:

- Different melanoma-related diagnostic labels (e.g., `MEL`) were mapped into a unified melanoma category.
- Various non-melanoma skin conditions were grouped into a common "unknown" or non-melanoma category.
- This preprocessing step simplified the classification task and allowed the model to focus on distinguishing melanoma from non-melanoma lesions.

Handling Missing Values: Missing values in the clinical metadata were handled during preprocessing through the `get_meta_data` function before dataset initialization:

- Missing values in features such as age, sex, and anatomical site were imputed using predefined default values or statistical rules.
- After imputation, metadata features were converted directly into tensors and supplied to the model.
- Missing categorical or numerical information was often encoded as a special value (e.g., -1), allowing the neural network to learn that the information was unavailable and appropriately reduce its influence during training.

Feature Extraction: Feature extraction was primarily performed automatically by deep convolutional neural networks:

- CNN architectures learned hierarchical visual representations directly from dermoscopic images.
- Training on detailed diagnostic categories rather than only binary labels encouraged the models to learn richer and more discriminative visual features.
- Clinical metadata features were also incorporated as an additional source of information.

Feature Engineering: Several feature engineering strategies were applied:

- Integration of image data and clinical metadata into a multimodal learning framework.
- Label engineering through diagnosis-based classification rather than simple benign/malignant labeling.
- Extensive image augmentation using the `Albumentations` library, including: Rotation; Horizontal and vertical flipping; Brightness adjustment.
- Combination of historical datasets (2018, 2019) with the 2020 competition dataset using `pd.concat` to increase data diversity and improve generalization.

Feature Selection: No explicit feature selection method was reported. The approach relied on deep learning models to automatically learn relevant features from both image and metadata inputs.
Modeling: The solution employed a large-scale ensemble of deep learning models:

- Multiple CNN architectures were used, including: **EfficientNet-B3 to EfficientNet-B7**; **SE-ResNeXt101**; **ResNeSt101**.
- Models were trained on both competition and external datasets from previous years.
- A multimodal architecture was supported through the `use_meta` option:
  - CNN branch for image feature extraction.
  - MLP branch for clinical metadata processing.
  - Fusion of image and metadata representations for final prediction.
- Diagnosis-based training was adopted, where models learned detailed diagnostic categories before producing melanoma predictions, resulting in an approximate AUC improvement of 0.01.
- The final solution leveraged architectural diversity and multimodal learning to improve robustness and predictive performance.

Source code available at: https://github.com/haqishen/SIIM-ISIC-Melanoma-Classification-1st-Place-Solution

b: Top 2

Data Cleaning: Several preprocessing steps were performed to improve label quality and dataset balance:

- The original binary classification problem was reformulated as a 3-class classification task consisting of: Melanoma, Benign nevus, Other lesions
- Since many images in the 2020 dataset lacked detailed diagnosis labels, a model trained on the 2019 dataset was used to infer nevus labels for previously "unknown" samples.
- A threshold-based relabeling strategy was applied using the 5th percentile of predictions on known nevus images.
- Severe class imbalance was addressed by upsampling melanoma samples from the 2020 dataset by a factor of 7.

Handling Missing Values: The dataset contained missing values in three metadata variables: `age_approx`, `sex`, `anatom_site_general_challenge`
The author applied a simple imputation strategy:

- Mean imputation for the numerical feature: `age_approx`
- Mode imputation for categorical features: `sex`, `anatom_site_general_challenge`

The author deliberately avoided extensive tuning of the imputation process to reduce the risk of overfitting to the training distribution.
Feature Extraction: Feature extraction was primarily performed through deep convolutional neural networks:

TABLE 11. Compare the solutions of Top 1 and Top 2.

AUTHOR	VAL.	FEATURE ENG.	MODEL	MISSING VAL.	PREPROCESS.
Top 1 Ha, Liu, Liu Private: 0.9490	5-Fold-CV: Data 2018+2019+2020, Validate on 2020	sex, age_approx 10 one-hot site image_size (bytes, log) n_images (log1p)	EfficientNet B3–B7 SE-ResNeXt101 ResNeSt101 CNN + Meta-branch MLP (512 → 128)	Sentinel: Sentinel + `dummy_na` anatom_site: One-hot encoding + `dummy_na=True` → create a separate `site_nan` column sex: male → 1, female → 0, NaN → -1 age_approx: normalized by 90, NaN → 0 Model interpretation: -1 = "missing information"	Label Standardization: Standardizing diagnosis labels (e.g., MEL, BKL...). Mapping 2019 → 2020 diagnosis. Data Augmentation: Transpose, Flip, Blur, CLAHE, Cutout, etc. Learning Rate Schedule: Cosine annealing with 1 warm-up epoch. Regularization: Multi-sample dropout (×5).
Top 2 Ian Pan Private: 0.9484	5-Fold-CV Triple stratified splits (@cdeotte) Validate on 2020 only	sex, age_approx, anatom_site Embedding 32-D for each feature Concat to final feature vector *The last model does not use metadata*	EfficientNet-B6 (512 × 512) EfficientNet-B7 (640 × 640) 3 class: melanoma / nevus / other Ensemble 3 5-fold models Pseudolabeling test set	Mean/Mode age_approx: imputed by **mean** sex, anatom_site: imputed by **mode** No category for NaN No extensively fine-tuning as risk of overfitting →non-metadata models achieved superior performance on final evaluation set	Relabel unknown→nevus/other by model 2019 Upsample 2020 melanoma ×7 RandAugment N=3, M∼Poisson(12) Square crop TTA (10 crops) AdamW + cosine warm restarts

- Visual features were automatically extracted from dermoscopic images using: **EfficientNet-B3 to EfficientNet-B8**, **SE-ResNeXt**, **ResNeSt**, **BiT-ResNet**.
- After experimentation, **EfficientNet-B6** and **EfficientNet-B7** provided the strongest performance and were retained for the final solution.
- Metadata features (`age`, `sex`, `anatomical location`) were also transformed into learned embedding representations.

Feature Engineering: Several feature engineering techniques were employed:

- Reformulation from binary classification to 3-class classification, encouraging the model to learn richer disease representations.
- Metadata embedding:
  - Age, sex, and anatomical location were each converted into 32-dimensional embedding vectors.
  - These embeddings were concatenated with CNN image features before classification.
- Extensive image augmentation using: **RandAugment (N=3)**; **Random square cropping**; **Test-Time Augmentation** (10 crops per image during inference)
- Pseudolabel generation: Soft 3-class predictions were generated for the test set using a pretrained 5-fold EfficientNet-B6 model. These pseudolabeled samples were incorporated into the training data for subsequent model retraining.
- Melanoma pseudolabeled samples with prediction scores greater than 0.5 were further upsampled by a factor of 7.

Feature Selection: No explicit feature selection method was applied. However, an important modeling decision was made regarding metadata:

- Metadata-enhanced models were initially explored.
- In the final stage, the author trained models without metadata, as these models achieved better private leaderboard performance and qualified for the competition's "without context" special prize.
- This effectively served as a form of feature exclusion based on empirical validation results.

Modeling: The final solution consisted of a large-scale deep learning ensemble:
Base Architectures

- EfficientNet-B6 (512 × 512 resolution)
- EfficientNet-B7 (640 × 640 resolution)
- Both models utilized Noisy Student pretraining weights.

Training Strategy

- Optimizer: AdamW
- Learning rate: 3×10^{-4}
- Scheduler: Cosine Annealing with Warm Restarts
- Snapshot ensembling:
  - B6: 3 snapshots (2 epochs per snapshot)
  - B7: 3 snapshots (3 epochs per snapshot)
- Validation:
  - 5-fold cross-validation
  - Triple-stratified splits
  - Validation performed exclusively on 2020 data

Semi-Supervised Learning

- A 5-fold EfficientNet-B6 model generated soft pseudolabels for the entire test set.
- The pseudolabeled test data was merged with the original training data and used to train an improved model from scratch.

Final Ensemble
The final prediction was obtained by blending three 5-fold model ensembles:

1) EfficientNet-B6 (512×512, without metadata)
2) EfficientNet-B7 (640×640, without metadata)
3) EfficientNet-B6 trained with pseudolabeled test data

Pseudolabeling was identified as the single most important contributor to the final leaderboard performance.

A crucial decision: dropping metadata at the final stage. This is the most notable point. The author acknowledges: "It wasn't until the last several days of the competition that I decided to train models without metadata, so I could be eligible for the without context special prize. It turns out that these models were actually my highest scoring private LB solutions!"

Reasons the author did not fine-tune the metadata "I did not want to spend too much time tuning this because I was afraid I would overfit to the distribution of the training set." He was concerned that over-tuning the metadata processing (including imputation) could cause the model to learn the distribution of the training set rather than the actual patterns—a risk that is especially dangerous when the test set has a different distribution

Source code available at: https://github.com/i-pan/kaggle-melanoma

4) Extracted Insights

The analysis of both competitions suggests that data quality and preprocessing strategies often contribute more to performance than model complexity alone. Effective handling of missing values, including treating missingness as an informative feature, consistently improved results. Additionally, feature engineering, data augmentation, and the incorporation of external data played a crucial role in enhancing model generalization. Both competitions also demonstrated that ensemble methods are highly effective for improving prediction robustness and leaderboard performance. Finally, top solutions emphasized avoiding overfitting through careful validation and restrained feature tuning, highlighting that additional features or metadata do not necessarily lead to better generalization on unseen data.

IV. CONCLUSION

The greatest value our team gained from this project is not merely the optimization of quantitative metrics, but a crystal-clear mindset regarding the true nature of data. We have developed a profound understanding of the "story" behind each feature, recognizing that every domain and feature group possesses its own distinct intrinsic meaning. By systematically applying Machine Learning techniques, we can extract truly meaningful signals. This enables us to drive model accuracy upward deliberately and explainably, rather than blindly chasing numbers.

Alongside empirical findings and the effective leverage of modern tools and technologies, the project yielded a vital qualitative lesson: the gap between academic theory, standardized tools (such as Scikit-learn), and real-world data processing demands immense flexibility over rigid rules. The clearest evidence of this is that despite our rigorous research into missing value imputation techniques from international papers, we still had to manually re-implement the source code to optimize it for our practical problem, instead of mechanically applying pre-existing frameworks.

V. APPENDICES

Source code available at: https://github.com/UIT24521104/Python-programming-for-machine-learning